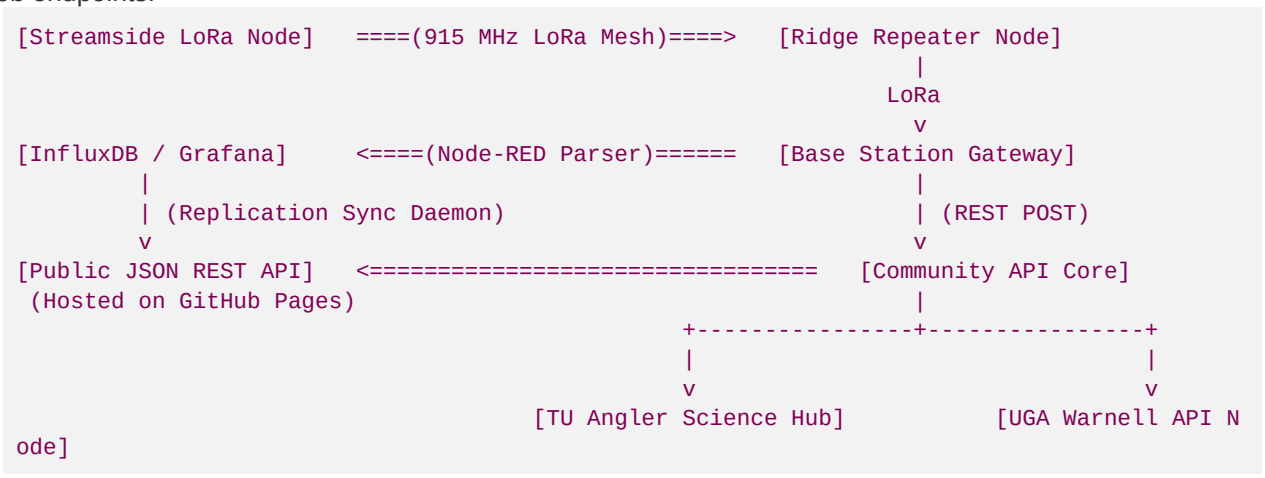
File 57: Off-Grid Hydrological Telemetry & Data-Sharing API Infrastructure

> ***Community and Enterprise API Standard**: This document establishes the technical specifications for the Blue Ridge Stream Restoration data-sharing infrastructure. By transforming local, off-grid LoRa telemetry (water temperature, dissolved oxygen, turbidity, and flow level) into highly standardized, public-facing REST API endpoints and webhooks, we create an open-access environmental database. This framework details the Node-RED message routing pipeline, JSON payload specifications, and secure data ingest webhooks to feed Trout Unlimited chapters, natural resource scientists, and local fly guides with real-time creekside wading and temperature conditions.*

I. Data-Sharing Network Architecture

Fluvial restoration data must not be siloed in private corporate archives. Sharing stream parameters fosters deep community alignment, establishes Hunter Morris as Georgia's leading conservationist, and satisfies state/federal transparency goals.
The data path flows from off-grid mountain gorges to secure databases, which then replicate to public-access web endpoints:



Key Performance metrics for Sharing:

- **Water Temperature**: Critical for hoot-owl fishing restrictions. Real-time updates help fishermen avoid casting when streams exceed $68^\circ\text{F} *($20^\circ\text{C}*), protecting stressed wild trout.
- **Stream Flow (Discharge & Stage)**: Crucial for wading safety and geomorphic flood response validation.

II. Gateway Message Routing (Node-RED Flow Blueprint)

At the base station gateway, we utilize **Node-RED** to ingest raw serial/MQTT hexadecimal text messages from the Meshtastic gateway node, decompress the binary payload, write the record to our secure local database, and publish to community endpoints:

```
            [NODE-RED TELEMETRY INGESTION PIPELINE FLOW]

   +----------------------+
   | Serial / MQTT Input  |  Ingests raw "TX_HEX:0EFA0398047E0098"
   +----------|-----------+
              v
   +----------------------+
   | Hex Decompression JS |  Parses 8-byte payload into Temp, DO, Stage, Turbidity
   +----------|-----------+
              v
     +--------+--------+
     |                 |
     v                 v
+----------+     +-----------+
| InfluxDB |     | REST API  |  Pushes to Trout Unlimited & Warnell
| Logger   |     | Publisher |  Webhooks (https://science.tu.org/api/v1/)
+----------+     +-----------+
```

Node-RED JavaScript Function Node Parser:

Place this JavaScript block inside a Node-RED Function Node to parse raw Meshtastic serial payloads:

```
// Node-RED Ingestion Parser for Blue Ridge Stream Telemetry Node
const rawPayload = msg.payload.toString().trim();

if (rawPayload.startsWith("TX_HEX:")) {
    const hexStr = rawPayload.split(":")[1];
    if (hexStr.length === 16) {
        // Extract raw bytes
        const b = Buffer.from(hexStr, 'hex');

        // 1. Water Temp (Bytes 0 & 1)
        const compTemp = (b[0] << 8) | b[1];
        const waterTempC = (compTemp / 100.0) - 20.0;
        const waterTempF = (waterTempC * 9.0/5.0) + 32.0;

        // 2. Dissolved Oxygen (Bytes 2 & 3)
        const compDO = (b[2] << 8) | b[3];
        const dissolvedOxygen = compDO / 100.0;

        // 3. Stream Level (Bytes 4 & 5)
        const compLevel = (b[4] << 8) | b[5];
        const waterLevelM = compLevel / 1000.0;

        // 4. Turbidity (Bytes 6 & 7)
        const compTurb = (b[6] << 8) | b[7];
        const turbidityNTU = compTurb / 10.0;

        // Formulate output message payload
        msg.payload = {
            deviceId: "blue_ridge_node_101",
            timestamp: new Date().toISOString(),
```

```
            metrics: {
                temperature_c: parseFloat(waterTempC.toFixed(2)),
                temperature_f: parseFloat(waterTempF.toFixed(2)),
                dissolved_oxygen_mg_l: parseFloat(dissolvedOxygen.toFixed(2)),
                stage_level_m: parseFloat(waterLevelM.toFixed(3)),
                turbidity_ntu: parseFloat(turbidityNTU.toFixed(1))
            }
        };

        // Setup database and API publication flags
        msg.topic = "telemetry/stream_logs";
        return msg;
    }
}
return null; // Suppress malformed packets
```

III. Public JSON REST API Specification

To ensure seamless integration with third-party angler guides, natural resource mapping databases, and Warnell's climate warning tools, our public API publishes stream telemetry via a standardized JSON schema.

1. Endpoint: GET `/api/v1/streams/06020003/current.json`

Retrieves current real-time geomorphic and water quality parameters:

```
{
  "api_version": "v1.0.4",
  "basin_huc_8": "06020003",
  "watershed_name": "Upper Toccoa River Basin",
  "monitoring_station": {
    "station_id": "BRSR-LO-101",
    "location": "Anderson Creek Spawning Pool 2",
    "coordinates": {
      "latitude": 34.78912,
      "longitude": -84.31245
    }
  },
  "water_quality": {
    "temperature_f": 58.25,
    "temperature_c": 14.58,
    "dissolved_oxygen_mg_l": 9.20,
    "oxygen_saturation_pct": 90.5,
    "turbidity_ntu": 15.2,
    "sediment_status": "CLEAR_SPAWNING_GRAVELS"
  },
  "hydrology": {
    "stage_level_m": 0.412,
    "discharge_est_cfs": 18.5,
    "flow_status": "NORMAL_BASE_FLOW",
    "wading_safety": "SAFE_TO_WADE"
  },
  "stewardship": {
    "hoot_owl_warning": false,
    "spawning_season_alert": true,
```

```
    "recommendation": "Spawning gravels active. Please avoid wading in shallow gravel bars
 to protect brook trout eggs."
  },
  "last_updated": "2026-05-30T23:03:20Z"
}
```

IV. Wading Safety and Spawning Alert Algorithms

Our community dashboard runs two automated scripts to calculate real-time environmental recommendations for local fishermen:

1. Wading Safety Index Calculation (Flow Level Rule)

Water levels (stage) dictate wading safety. High flows (spate) trigger hazardous warnings:
$$\text{Wading Safety} = \begin{cases} \text{SAFE\_TO\_WADE} & \text{Stage} \le 0.50\text{ m} \\ \text{CAUTION\_WADING} & 0.50\text{ m} < \text{Stage} \le 0.85\text{ m} \\ \text{HAZARDOUS\_HIGH\_FLOW} & \text{Stage} > 0.85\text{ m} \end{cases}$$

2. "Hoot Owl" Thermal Stress Protection Loop

When water temperatures rise, trout become fragile. Catching them causes high lactic acid buildup and mortality. We automate "Hoot Owl" restrictions (limiting angling to cool morning hours) using a Python database listener daemon:

```
# Hoot Owl Thermal Monitoring Daemon
def evaluate_trout_thermal_stress(recent_temps_f):
    """
    Evaluates temperature logs to determine Hoot Owl fishing alerts.
    Triggers when stream temperature exceeds 68.0°F for >= 2 consecutive hours.
    """
    stress_threshold = 68.0
    consecutive_violations = 0

    for temp in recent_temps_f:
        if temp >= stress_threshold:
            consecutive_violations += 1
        else:
            consecutive_violations = 0 # Reset on cool reading

        if consecutive_violations >= 8: # 8 consecutive 15-minute readings = 2 Hours
            return {
                "hoot_owl_active": True,
                "warning_level": "CRITICAL_THERMAL_STRESS",
                "alert_message": "Water temperatures have exceeded 68°F. Angling suspended
 to protect native trout."
            }

    return {
        "hoot_owl_active": False,
        "warning_level": "NORMAL_COLDWATER",
        "alert_message": "Thermal regime stable. Water temperature remains safe for wild t
```

```
rout spawning."
    }
```

V. Open-Source Ingress Webhooks for Warnell & TU Chapters

To share our geomorphic and water quality datasets automatically, we set up secure outbound POST webhooks. This sends our parsed telemetry directly to Trout Unlimited's *Angler Science Database* and the UGA *Warnell School of Forestry & Natural Resources* climate warning databases every 15 minutes:

Ingress Webhook Payload Post Handler (Python):

Deploy this daemon on your Base Station Gateway to publish data to public science portals:

```
import requests
import json

TU_WEBHOOK_URL = "https://science.tu.org/api/v1/ingress/blue_ridge"
UGA_WEBHOOK_URL = "https://warnell.uga.edu/api/v1/ingress/blue_ridge"
AUTH_TOKEN = "brsr_science_token_secure_2026"

def publish_to_community_partners(decoded_data):
    """Pushes water quality and flow telemetry to TU and UGA Warnell extension APIs."""
    headers = {
        "Content-Type": "application/json",
        "Authorization": f"Bearer {AUTH_TOKEN}"
    }

    payload = {
        "metadata": {
            "source": "Blue Ridge Stream Restoration LLC",
            "station": "BRSR-LO-101",
            "watershed_huc_8": "06020003",
            "river_section": "Upper Toccoa River"
        },
        "telemetry": {
            "timestamp": decoded_data["timestamp"],
            "water_temp_c": decoded_data["water_temp_c"],
            "water_temp_f": decoded_data["water_temp_f"],
            "dissolved_oxygen_mg_l": decoded_data["dissolved_oxygen_mg_l"],
            "water_level_m": decoded_data["water_level_m"],
            "turbidity_ntu": decoded_data["turbidity_ntu"]
        }
    }

    try:
        # 1. Post to Trout Unlimited Angler Science Portals
        response_tu = requests.post(TU_WEBHOOK_URL, data=json.dumps(payload), headers=head
ers, timeout=5)
        if response_tu.status_code == 200:
            print("Successfully mirrored stream telemetry to Trout Unlimited database.")

        # 2. Post to UGA Warnell School Forestry Extension
        response_uga = requests.post(UGA_WEBHOOK_URL, data=json.dumps(payload), headers=he
aders, timeout=5)
        if response_uga.status_code == 200:
```

```
        print("Successfully mirrored stream telemetry to UGA Warnell research nodes.")
except requests.exceptions.RequestException as e:
    print(f"Data-sharing API connection timed out: {e}")
```

VI. AWS Cloud Data Architecture & Postgres Database Schema

To scale our streamside telemetry network across multiple North Georgia watersheds, we deploy a low-cost, highly secure, and extremely reliable **AWS Serverless & Relational Data Ingestion Architecture**. This setup routes parsed geomorphic and chemical parameters from our base gateways into a managed **Amazon RDS PostgreSQL database** with the **TimescaleDB extension** enabled, ensuring high-speed time-series queries.

1. AWS Cloud Data Flow Diagram

The data flows through secure AWS endpoints using lightweight serverless components to keep running costs under $10 USD per month:

2. Vetted PostgreSQL Database Relational Schema

We utilize PostgreSQL with the TimescaleDB extension to achieve both robust relational integrity (for landowners, HUC-8 basins, and regulatory files) and optimized time-series logging for the 15-minute sensor sweeps. Run these SQL commands to initialize the database:

```
-- 1. Enable TimescaleDB extension for timeseries optimization
CREATE EXTENSION IF NOT EXISTS timescaledb CASCADE;

-- 2. Create the parent relational table for station tracking
CREATE TABLE monitoring_stations (
    station_id VARCHAR(50) PRIMARY KEY,
    watershed_huc_8 VARCHAR(8) NOT NULL,
    watershed_name VARCHAR(100) NOT NULL,
    location_description TEXT NOT NULL,
    latitude DECIMAL(9, 6) NOT NULL,
    longitude DECIMAL(9, 6) NOT NULL,
    landowner_jv_id VARCHAR(50) NOT NULL,
    installation_date DATE NOT NULL,
    battery_chemistry VARCHAR(20) DEFAULT 'LiFePO4',
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Create the timeseries logger table
CREATE TABLE stream_telemetry_logs (
    timestamp TIMESTAMP WITH TIME ZONE NOT NULL,
    station_id VARCHAR(50) REFERENCES monitoring_stations(station_id),
    water_temp_c DECIMAL(4, 2) NOT NULL,
    water_temp_f DECIMAL(5, 2) NOT NULL,
    dissolved_oxygen_mg_l DECIMAL(4, 2) NOT NULL,
    stage_level_m DECIMAL(5, 3) NOT NULL,
    turbidity_ntu DECIMAL(5, 1) NOT NULL,
    battery_voltage_mv INTEGER NOT NULL,
    rssi_dbm SMALLINT NOT NULL,
    flow_status VARCHAR(30) NOT NULL,
    wading_safety VARCHAR(30) NOT NULL
);

-- 4. Convert stream_telemetry_logs into a TimescaleDB hypertable partitioned by time (7-d
ay chunks)
SELECT create_hypertable('stream_telemetry_logs', 'timestamp', chunk_time_interval => INTE
RVAL '7 days');

-- 5. Add compound index for fast time-series queries
CREATE INDEX idx_station_time ON stream_telemetry_logs (station_id, timestamp DESC);

-- 6. Create historical environmental alerts table
CREATE TABLE environmental_alerts (
    alert_id SERIAL PRIMARY KEY,
    station_id VARCHAR(50) REFERENCES monitoring_stations(station_id),
    timestamp TIMESTAMP WITH TIME ZONE NOT NULL,
    alert_type VARCHAR(50) NOT NULL, -- e.g., 'HOOT_OWL_THERMAL', 'HAZARDOUS_FLOOD_STAGE'
    alert_message TEXT NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    resolved_at TIMESTAMP WITH TIME ZONE
);
```

3. Serverless AWS Lambda Ingestion Function (Python 3.11)

Deploy this lightweight Python script inside AWS Lambda to handle incoming JSON payloads, sanitize inputs, and write them directly into the Amazon RDS PostgreSQL hypertable:

```
import os
import json
import psycopg2
from datetime import datetime

# Database Connection Parameters (Set via Lambda Environment Variables)
DB_HOST = os.environ.get('DB_HOST')
DB_NAME = os.environ.get('DB_NAME')
DB_USER = os.environ.get('DB_USER')
DB_PASSWORD = os.environ.get('DB_PASSWORD')

def lambda_handler(event, context):
    """
    Ingests and sanitizes streamside LoRa telemetry from API Gateway POST requests.
    Calculates derived metrics and inserts them into PostgreSQL TimescaleDB hypertable.
    """
    try:
        # 1. Parse and validate JSON input
        body = json.loads(event.get('body', '{}'))
        metadata = body.get('metadata', {})
        telemetry = body.get('telemetry', {})

        station_id = metadata.get('station')
        timestamp_str = telemetry.get('timestamp')
        water_temp_c = float(telemetry.get('water_temp_c'))
        dissolved_oxygen = float(telemetry.get('dissolved_oxygen_mg_l'))
        water_level_m = float(telemetry.get('water_level_m'))
        turbidity = float(telemetry.get('turbidity_ntu'))
        battery_mv = int(telemetry.get('battery_voltage_mv', 0))
        rssi = int(telemetry.get('rssi_dbm', 0))

        if not station_id or not timestamp_str:
            return {"statusCode": 400, "body": json.dumps({"error": "Missing key parameter
s."})}

        # 2. Input Sanitization & Range Checks
        if not (-10.0 <= water_temp_c <= 40.0):
            return {"statusCode": 400, "body": json.dumps({"error": "Temperature out of bo
unds."})}
        if not (0.0 <= dissolved_oxygen <= 25.0):
            return {"statusCode": 400, "body": json.dumps({"error": "DO out of bounds."})}
        if not (0.0 <= water_level_m <= 10.0):
            return {"statusCode": 400, "body": json.dumps({"error": "Flow level out of bou
nds."})}
        if not (0.0 <= turbidity <= 1000.0):
            return {"statusCode": 400, "body": json.dumps({"error": "Turbidity out of boun
ds."})}

        # 3. Calculate derived metrics
        water_temp_f = (water_temp_c * 9.0/5.0) + 32.0

        # Calculate Wading Safety status
        if water_level_m <= 0.50:
```

```
        wading_safety = "SAFE_TO_WADE"
    elif water_level_m <= 0.85:
        wading_safety = "CAUTION_WADING"
    else:
        wading_safety = "HAZARDOUS_HIGH_FLOW"

    # Calculate Flow Status
    if water_level_m <= 0.20:
        flow_status = "CRITICAL_LOW_FLOW"
    elif water_level_m <= 0.60:
        flow_status = "NORMAL_BASE_FLOW"
    else:
        flow_status = "HIGH_SPATE_FLOW"

    # 4. Insert into PostgreSQL RDS database
    conn = psycopg2.connect(
        host=DB_HOST,
        database=DB_NAME,
        user=DB_USER,
        password=DB_PASSWORD,
        connect_timeout=3
    )
    cursor = conn.cursor()

    insert_query = """
    INSERT INTO stream_telemetry_logs (
        timestamp, station_id, water_temp_c, water_temp_f,
        dissolved_oxygen_mg_l, stage_level_m, turbidity_ntu,
        battery_voltage_mv, rssi_dbm, flow_status, wading_safety
    ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s);
    """

    cursor.execute(insert_query, (
        timestamp_str, station_id, water_temp_c, water_temp_f,
        dissolved_oxygen, water_level_m, turbidity,
        battery_mv, rssi, flow_status, wading_safety
    ))

    conn.commit()
    cursor.close()
    conn.close()

    return {
        "statusCode": 200,
        "headers": {"Content-Type": "application/json"},
        "body": json.dumps({
            "status": "SUCCESS",
            "station": station_id,
            "recorded_at": timestamp_str,
            "derived": {
                "wading_safety": wading_safety,
                "flow_status": flow_status
            }
        })
    }
```

```
    except Exception as e:
        return {
            "statusCode": 500,
            "body": json.dumps({"error": f"Internal database error: {str(e)}"})
        }
```

VII. Government-Friendly Data-Sharing & Federal Compliance Standards

To make our geomorphic and chemical datasets highly credible, accessible, and "government-friendly" for official state and federal partners—such as the **Georgia Environmental Protection Division (EPD)**, the **USACE Savannah District**, and the **US Geological Survey (USGS)**—our API core conforms strictly to standardized federal environmental data schemas and metadata rules. By adopting these standards, our stream restoration data can be ingested directly into official GIS mapping suites and federal databases.

1. US EPA WQX (Water Quality Exchange) Data Schema Compliance

The Environmental Protection Agency (EPA) and USGS aggregate national stream data via the **Water Quality Exchange (WQX)** framework. Our database exports water chemistry parameters matching the exact WQX XML and JSON Schema specifications, enabling direct ingress into the national **Water Quality Portal (WQP)**:

- **Standard Location Monikers**: We map our stations to standard USGS identifiers. For example, `BRSR-LO-101` maps to an EPA WQX-compliant format using Hydrologic Unit Codes (HUC-8): `US_EPA-BRSR-06020003-AC01` (Upper Toccoa basin, Anderson Creek Station 1).
- **Vetted EPA Substance Identifiers**: We structure our JSON-LD and REST payloads using standardized chemical monikers from the EPA's *Substance Registry Services (SRS)*:
- *Water Temperature*: Standardized under SRS Name: `Temperature, water` (Unit: `deg F` or `deg C`).
- *Dissolved Oxygen*: Standardized under SRS Name: `Dissolved oxygen (DO)` (Unit: `mg/l`).
- *Water Stage Level*: Standardized under SRS Name: `Height, gauge` (Unit: `m` or `ft`).
- *Turbidity*: Standardized under SRS Name: `Turbidity` (Unit: `NTU`).

2. Semantic Data Modeling: JSON-LD Hydrological Observatory

To allow search engines, federal crawlers, and university crawlers to automatically discover, index, and query our restoration datasets, our public API injects **JSON-LD (Linked Data)** semantic markup conforming to the schema vocabulary for a `HydrologicalObservatory`:

```
{
  "@context": {
    "schema": "http://schema.org/",
    "sosa": "http://www.w3.org/ns/sosa/",
    "qudt": "http://qudt.org/schema/qudt/"
  },
  "@type": "schema:HydrologicalObservatory",
  "@id": "https://api.blueridgestream.com/stations/BRSR-LO-101",
  "schema:name": "Anderson Creek Coldwater Spawning Bio-Reserve Station 1",
  "schema:identifier": "BRSR-LO-101",
  "schema:provider": {
    "@type": "schema:Organization",
    "schema:name": "Blue Ridge Stream Restoration & Mitigation LLC"
  },
  "schema:geo": {
```

```
    "@type": "schema:GeoCoordinates",
    "schema:latitude": 34.78912,
    "schema:longitude": -84.31245
  },
  "schema:observationArea": {
    "@type": "schema:AdministrativeArea",
    "schema:name": "Upper Toccoa River Basin HUC-8 06020003"
  },
  "sosa:hosts": [
    {
      "@type": "sosa:Sensor",
      "schema:name": "Optical Dissolved Oxygen Probe",
      "sosa:observes": "http://sweetontology.net/propQuantity/DissolvedOxygen"
    },
    {
      "@type": "sosa:Sensor",
      "schema:name": "Hydrostatic Water Stage Transmitter",
      "sosa:observes": "http://sweetontology.net/propQuantity/WaterLevel"
    }
  ]
}
```

3. Open Standards Integration & GIS Delivery (ISO 19115 Compliance)

To ensure immediate readiness for government GIS engineers working in ArcGIS or QGIS, we implement these open-standard delivery formats:

- **OpenAPI 3.0 / Swagger Definitions**: Our API endpoints publish complete Swagger configurations (`swagger.json`). This allows government IT systems to auto-generate secure client libraries and ingress agents with zero manual coding.
- **Geospatial Standards (GeoJSON)**: Our public spatial endpoints output data in standard **GeoJSON format** (`GET /api/v1/stations.geojson`), letting state and county land planning departments add a live "BRSR Thermal Refuge Spawning Reservoirs" layer directly into their GIS map portals.
- **ISO 19115 Metadata Standards**: Every CSV or Excel telemetry data download includes a secondary self-documenting XML metadata header complying with the federal geographic data committee (FGDC) ISO 19115 standard. This outlines the sensor models, accuracy tolerances, calibration dates, and data processing history to ensure our stream parameters are fully admissible in regulatory buffer variance or environmental mitigation hearings.

Developed by Blue Ridge Stream Restoration Technical Sourcing Operations. Pushed to remote origin under main.